Technical Questions & Answers

---

1) **How do you decide whether a performance issue is application-side, dependency-side, or workload model related?**

Answer:

To determine the root cause of a performance issue, I follow a structured approach by isolating where the degradation occurs:

- **Application-side issue:**
  - All or most endpoints show increased latency or errors
  - CPU or memory usage may be high
  - Response times degrade consistently across the system
- **Dependency-side issue:**
  - Only specific endpoints are affected
  - These endpoints typically rely on external systems (e.g., database, authentication service)
  - Other endpoints remain stable
- **Workload model issue:**
  - Problems appear only under specific load patterns
  - For example, high concurrency with limited test data may cause session conflicts
  - Results improve when adjusting data, pacing, or concurrency

In this assessment:

- The issue was initially **workload-related (shared user data)**
- After fixing data, the system behaved correctly, confirming no major application-side issue

---

2) **Why are percentiles and throughput both important in performance reporting?**

Answer:

Both metrics provide different but complementary insights:

- **Percentiles (P90, P95, P99):**
  - Represent user experience

    - Show how slow the system is for most users
    - For example, P95 means 95% of users experienced response times below that value
- **Throughput:**
    - Measures system capacity (requests per second)
    - Indicates how much load the system can handle

Using only averages can hide slow requests, so percentiles are critical.
Using only percentiles ignores system capacity, so throughput is also required.

Together, they provide a complete view of:

- performance quality (user experience)
- system capability (load handling)

---

3) **What is correlation? Give one practical example from your solution.**

Answer:

Correlation is the process of extracting dynamic data from a response and reusing it in subsequent requests.

It is essential when dealing with:

- authentication tokens
- session IDs
- dynamic values generated by the server

Example from this solution:

After login, the system returns an accessToken, which must be used in later requests.

```
jsonPath("$.accessToken").saveAs("token");
```

This value is then reused in the next request:

```
.header("Authorization", "Bearer #{token}")
```

Without correlation:

- requests would fail
- the script would not simulate real user behavior

---

4) Why can repeated test runs produce different outcomes even with the same script?

Answer:

Performance test results may vary due to several factors:

- **Shared environment (especially public APIs)**
- **Network variability**
- **Caching effects**
- **Thread scheduling and timing differences**
- **Concurrent usage by other users**
- **Test data reuse or conflicts**

In this assessment:

- Initial runs showed higher failures due to shared user reuse
- After increasing unique users, results became more stable

This demonstrates that even with the same script, external factors and data design can significantly impact results.

---

5) What would you investigate first if latency increases sharply but CPU and memory remain normal?

Answer:

If latency increases while CPU and memory remain stable, the issue is likely not resource-related but I/O or dependency-related.

The investigation should focus on:

1. **External dependencies**
    - Database queries
    - Third-party APIs
    - authentication services
2. **Connection limits**
    - exhausted connection pools
    - slow connection acquisition
3. **Thread pools / request queuing**

    - threads waiting for resources
    - increased queue time
4. **Network latency**
    - slow communication between services
    - DNS or routing delays
5. **Rate limiting or throttling**
    - system delaying or rejecting requests